

Microsoft Fabric Warehouse

T-SQL DMV Monitoring Guide

Database Administration — Technical Reference

April 2025 | Version 1.0

Table of Contents

1.	Overview of DMVs in Fabric Warehouse	3
2.	DMV Access & Permissions	3
3.	Live Connection & Session Monitoring	4
4.	Active Request Monitoring	5
5.	Query Insights Schema	6
5.1.	exec_requests_history	6
5.2.	exec_sessions_history	7
5.3.	long_running_queries	7
5.4.	frequently_run_queries	8
6.	Transaction & Lock Monitoring	8
7.	Killing & Managing Sessions	9
8.	Performance Diagnostics Queries	9
9.	Billing & Capacity Utilisation	10
10.	DMV Quick Reference Card	11

1. Overview of DMVs in Fabric Warehouse

Dynamic Management Views (DMVs) are system-level SQL views that expose internal operational state of the Fabric Warehouse engine. They allow DBAs and data engineers to inspect live connections, running queries, transaction locks, and historical performance data — all via standard T-SQL without any additional tooling.

Fabric Warehouse automatically provisions two categories of monitoring views:

- System DMVs (sys.dm_exec_*) — for real-time session and request state.
- Query Insights Schema (queryinsights.*) — for historical and aggregated query analytics.

■ **Note:** All DMV queries must be run against the Fabric Warehouse SQL Endpoint using T-SQL. Tools like SSMS, Azure Data Studio, or the Fabric portal query editor are supported.

2. DMV Access & Permissions

Access to DMVs in Fabric Warehouse is role-based. Different workspace roles grant different levels of visibility:

Workspace Role	DMVs Accessible	KILL Permission
Viewer	sys.dm_exec_sessions, sys.dm_exec_requests	No
Contributor	sys.dm_exec_sessions, sys.dm_exec_requests	No
Member	sys.dm_exec_sessions, sys.dm_exec_requests	No
Admin	All DMVs — full access	Yes

To grant VIEW DATABASE STATE to a specific user (for non-admin DMV access):

■ Grant DMV Permission

```
-- Run in the context of the Fabric Warehouse database
GRANT VIEW DATABASE STATE TO [user@yourdomain.com];
```

3. Live Connection & Session Monitoring

Use **sys.dm_exec_connections** to inspect all active connections to the Fabric Warehouse SQL endpoint, including client details and protocol information.

■ All Active Connections

```
SELECT
    session_id,
    client_net_address,
    client_tcp_port,
    auth_scheme,
    protocol_type,
    connect_time
FROM sys.dm_exec_connections
ORDER BY connect_time DESC;
```

Use **sys.dm_exec_sessions** to see all active sessions including idle ones, their resource consumption, and query text:

■ Active Sessions with Resource Usage

```
SELECT
    s.session_id,
    s.login_name,
    s.status,
    s.cpu_time,
    s.memory_usage,
    s.total_elapsed_time / 1000.0 AS elapsed_sec,
    s.last_request_start_time,
    s.program_name
FROM sys.dm_exec_sessions s
WHERE s.is_user_process = 1
ORDER BY s.total_elapsed_time DESC;
```

■ Sessions Joined with Connection Info

```
SELECT
    s.session_id,
    s.login_name,
    c.client_net_address,
    c.protocol_type,
    s.status,
    s.cpu_time,
    s.total_elapsed_time / 1000.0 AS elapsed_sec
FROM sys.dm_exec_sessions s
JOIN sys.dm_exec_connections c ON s.session_id = c.session_id
WHERE s.is_user_process = 1;
```

4. Active Request Monitoring

sys.dm_exec_requests shows all currently executing or waiting requests. This is the primary DMV for identifying long-running or blocked queries in real time.

■ All Currently Running Requests

```
SELECT
  r.session_id,
  r.status,
  r.command,
  r.cpu_time,
  r.total_elapsed_time / 1000.0 AS elapsed_sec,
  r.wait_type,
  r.wait_time / 1000.0 AS wait_sec,
  r.blocking_session_id,
  r.logical_reads,
  r.writes
FROM sys.dm_exec_requests r
WHERE r.session_id > 50 -- exclude system sessions
ORDER BY r.total_elapsed_time DESC;
```

■ Long-Running Requests (over 30 seconds)

```
SELECT
  r.session_id,
  s.login_name,
  r.status,
  r.total_elapsed_time / 1000.0 AS elapsed_sec,
  r.wait_type,
  r.blocking_session_id
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
WHERE r.total_elapsed_time > 30000 -- 30 seconds in ms
AND r.session_id > 50
ORDER BY r.total_elapsed_time DESC;
```

■ Find Blocking Chains

```
SELECT
blocking.session_id AS blocker_session_id,
blocking_s.login_name AS blocker_login,
blocked.session_id AS blocked_session_id,
blocked_s.login_name AS blocked_login,
blocked.wait_type,
blocked.wait_time / 1000.0 AS blocked_wait_sec
FROM sys.dm_exec_requests blocked
JOIN sys.dm_exec_requests blocking
ON blocked.blocking_session_id = blocking.session_id
JOIN sys.dm_exec_sessions blocked_s ON blocked.session_id = blocked_s.session_id
JOIN sys.dm_exec_sessions blocking_s ON blocking.session_id = blocking_s.session_id;
```

5. Query Insights Schema

When a Fabric Warehouse is provisioned, a **queryinsights** schema is automatically created. It contains four views for historical and aggregated query analysis — a powerful tool for proactive performance management.

View Name	Purpose
queryinsights.exec_requests_history	Full history of completed/failed/cancelled queries
queryinsights.exec_sessions_history	Historical session-level resource consumption
queryinsights.long_running_queries	Aggregated stats for slow-running statements
queryinsights.frequently_run_queries	Most repeated queries ranked by execution count

5.1 exec_requests_history

Provides a granular record of every completed query — including status, row counts, elapsed time, and the full SQL text.

■ Recent Query History (last 50)
<pre>SELECT TOP 50 distributed_statement_id, start_time, end_time, total_elapsed_time_ms / 1000.0 AS elapsed_sec, status, row_count, login_name, command FROM queryinsights.exec_requests_history ORDER BY start_time DESC;</pre>
■ Failed Queries in Last 24 Hours
<pre>SELECT distributed_statement_id, login_name, start_time, total_elapsed_time_ms / 1000.0 AS elapsed_sec, status, command FROM queryinsights.exec_requests_history WHERE status IN ('Failed', 'Cancelled') AND start_time >= DATEADD(HOUR, -24, GETUTCDATE()) ORDER BY start_time DESC;</pre>

5.2 exec_sessions_history

■ Session History — Resource Usage per User

```
SELECT
  login_name,
  COUNT(*) AS total_sessions,
  SUM(cpu_time_ms) / 1000.0 AS total_cpu_sec,
  SUM(total_elapsed_time_ms) / 1000.0 AS total_elapsed_sec,
  MAX(memory_usage_mb) AS peak_memory_mb
FROM queryinsights.exec_sessions_history
GROUP BY login_name
ORDER BY total_cpu_sec DESC;
```

5.3 long_running_queries

Aggregates median, average, min, and max execution times per query. Ideal for identifying recurring performance offenders.

■ Top 20 Slow Queries by Median Duration

```
SELECT TOP 20
  command,
  median_total_elapsed_time_ms / 1000.0 AS median_sec,
  avg_total_elapsed_time_ms / 1000.0 AS avg_sec,
  max_total_elapsed_time_ms / 1000.0 AS max_sec,
  min_total_elapsed_time_ms / 1000.0 AS min_sec,
  last_run_total_elapsed_time_ms / 1000.0 AS last_run_sec,
  number_of_iterations AS run_count
FROM queryinsights.long_running_queries
ORDER BY median_total_elapsed_time_ms DESC;
```

5.4 frequently_run_queries

■ Most Frequently Executed Queries

```
SELECT TOP 20
  command,
  number_of_iterations AS execution_count,
  avg_total_elapsed_time_ms / 1000.0 AS avg_sec,
  max_total_elapsed_time_ms / 1000.0 AS max_sec
FROM queryinsights.frequently_run_queries
ORDER BY number_of_iterations DESC;
```


6. Transaction & Lock Monitoring

Fabric Warehouse uses **Snapshot Isolation** as its only supported isolation level. Locks are held at the table level. INSERTs are rarely blocked (Parquet files are immutable), but concurrent UPDATES and DELETES can conflict.

■ **Note:** If two sessions run UPDATE on the same table simultaneously, the session that commits last will fail with a write-write conflict. Design ETL pipelines to avoid concurrent DML on the same table.

■ View Active Transactions

```
SELECT
transaction_id,
session_id,
name AS transaction_name,
transaction_begin_time,
DATEDIFF(SECOND, transaction_begin_time, GETUTCDATE()) AS age_sec,
transaction_type,
transaction_state
FROM sys.dm_tran_active_transactions
ORDER BY transaction_begin_time;
```

■ Detect Long-Running Open Transactions

```
SELECT
t.session_id,
s.login_name,
t.transaction_id,
t.transaction_begin_time,
DATEDIFF(SECOND, t.transaction_begin_time, GETUTCDATE()) AS age_sec
FROM sys.dm_tran_session_transactions t
JOIN sys.dm_exec_sessions s ON t.session_id = s.session_id
WHERE DATEDIFF(SECOND, t.transaction_begin_time, GETUTCDATE()) > 60
ORDER BY age_sec DESC;
```

7. Killing & Managing Sessions

Only **Workspace Admins** can issue the KILL command. Use this to terminate runaway queries or free up blocked resources.

■ Identify a Session to Kill

```
-- Step 1: Find the session_id
SELECT session_id, login_name, status,
total_elapsed_time / 1000.0 AS elapsed_sec,
wait_type
FROM sys.dm_exec_requests
WHERE session_id > 50
ORDER BY total_elapsed_time DESC;
```

■ Terminate a Specific Session

```
-- Step 2: Kill the session (replace 123 with actual session_id)
KILL 123;
```

■ **Note:** Always verify the session belongs to a genuine problem query before issuing KILL. The command immediately rolls back any open transaction for that session.

8. Performance Diagnostics Queries

■ CPU-Intensive Queries from History

```
SELECT TOP 10
login_name,
start_time,
total_elapsed_time_ms / 1000.0 AS elapsed_sec,
row_count,
status,
command
FROM queryinsights.exec_requests_history
WHERE status = 'Succeeded'
ORDER BY total_elapsed_time_ms DESC;
```

■ Query Volume by Hour (Last 7 Days)

```
SELECT
DATEPART(HOUR, start_time) AS hour_of_day,
COUNT(*) AS query_count,
AVG(total_elapsed_time_ms)/1000.0 AS avg_elapsed_sec,
MAX(total_elapsed_time_ms)/1000.0 AS max_elapsed_sec
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY, -7, GETUTCDATE())
GROUP BY DATEPART(HOUR, start_time)
ORDER BY hour_of_day;
```

■ Per-User Query Performance Summary

```
SELECT
  login_name,
  COUNT(*) AS total_queries,
  SUM(CASE WHEN status='Failed'
    THEN 1 ELSE 0 END) AS failed_queries,
  AVG(total_elapsed_time_ms)/1000.0 AS avg_elapsed_sec,
  MAX(total_elapsed_time_ms)/1000.0 AS max_elapsed_sec,
  SUM(row_count) AS total_rows_returned
FROM queryinsights.exec_requests_history
GROUP BY login_name
ORDER BY total_queries DESC;
```

9. Billing & Capacity Utilisation

Fabric Warehouse compute is measured in **Capacity Units (CUs)**. Monitoring CU consumption helps admins manage costs and avoid throttling. The primary tool for this is the **Fabric Capacity Metrics App** available in the Power BI admin portal.

■ Query Duration Distribution (Workload Profiling)

```
SELECT
CASE
WHEN total_elapsed_time_ms < 1000 THEN '< 1s'
WHEN total_elapsed_time_ms < 5000 THEN '1-5s'
WHEN total_elapsed_time_ms < 30000 THEN '5-30s'
WHEN total_elapsed_time_ms < 60000 THEN '30-60s'
ELSE '> 60s'
END AS duration_bucket,
COUNT(*) AS query_count,
ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 1) AS pct
FROM queryinsights.exec_requests_history
WHERE status = 'Succeeded'
GROUP BY
CASE
WHEN total_elapsed_time_ms < 1000 THEN '< 1s'
WHEN total_elapsed_time_ms < 5000 THEN '1-5s'
WHEN total_elapsed_time_ms < 30000 THEN '5-30s'
WHEN total_elapsed_time_ms < 60000 THEN '30-60s'
ELSE '> 60s'
END
ORDER BY MIN(total_elapsed_time_ms);
```

10. DMV Quick Reference Card

DMV / View	Category	Key Use Case
sys.dm_exec_connections	Live	Active connections & client info
sys.dm_exec_sessions	Live	Session resource usage & status
sys.dm_exec_requests	Live	Currently running queries
sys.dm_tran_active_transactions	Live	Open transactions & age
sys.dm_tran_session_transactions	Live	Session-to-transaction mapping
queryinsights.exec_requests_history	Historical	All completed/failed query records
queryinsights.exec_sessions_history	Historical	Per-session resource history
queryinsights.long_running_queries	Aggregated	Slow query patterns over time
queryinsights.frequently_run_queries	Aggregated	High-frequency query identification

Best Practices for DMV-Based Monitoring

- Schedule regular polling of exec_requests_history to build a performance baseline.
- Use long_running_queries weekly to identify optimisation candidates.
- Monitor blocked sessions daily using sys.dm_exec_requests (blocking_session_id IS NOT NULL).
- Alert on failed query counts exceeding a threshold via queryinsights.exec_requests_history.
- Restrict KILL permission to Workspace Admin role — never grant it to developers.
- Cross-reference DMV data with the Fabric Capacity Metrics App for cost attribution.